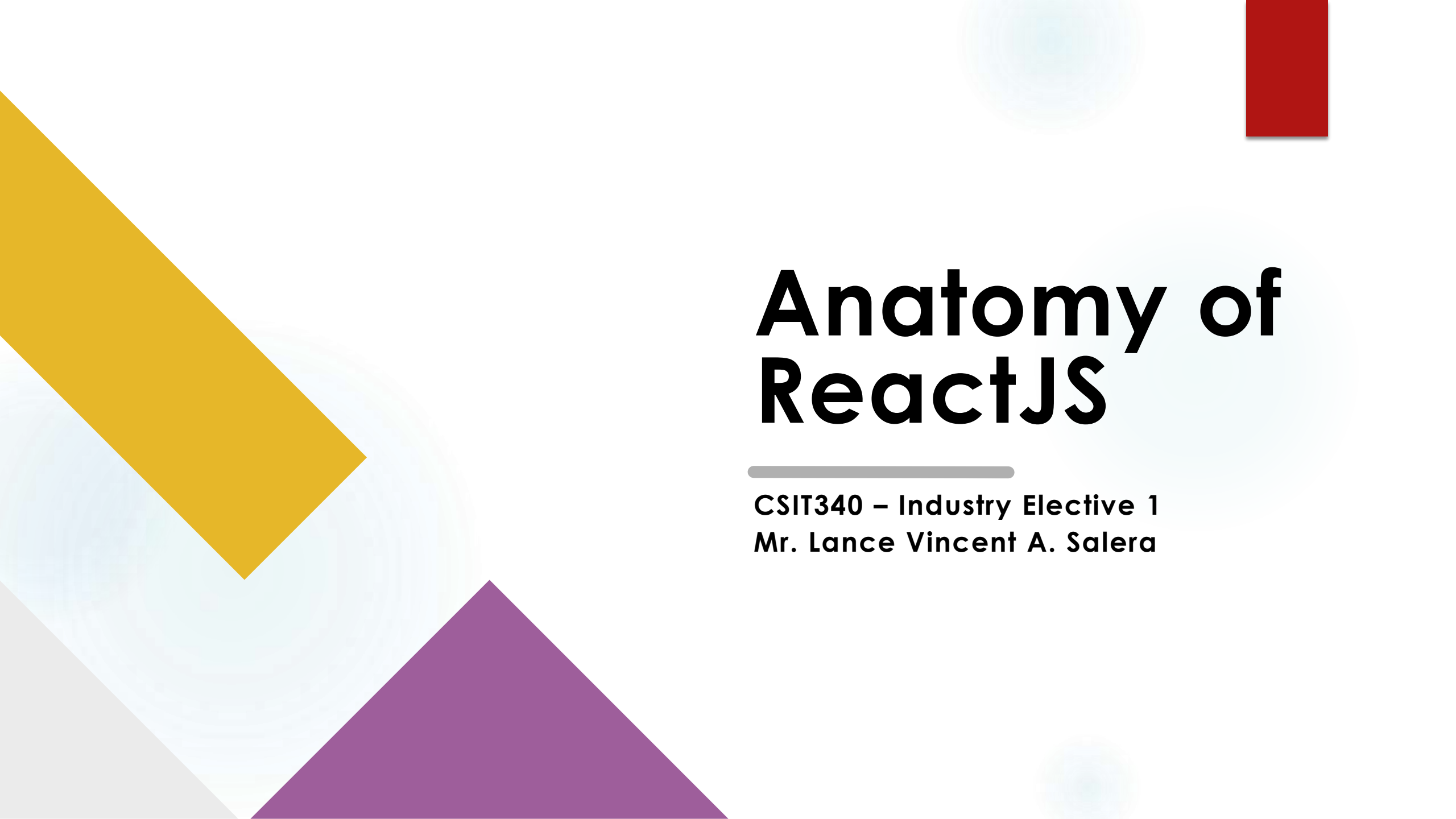




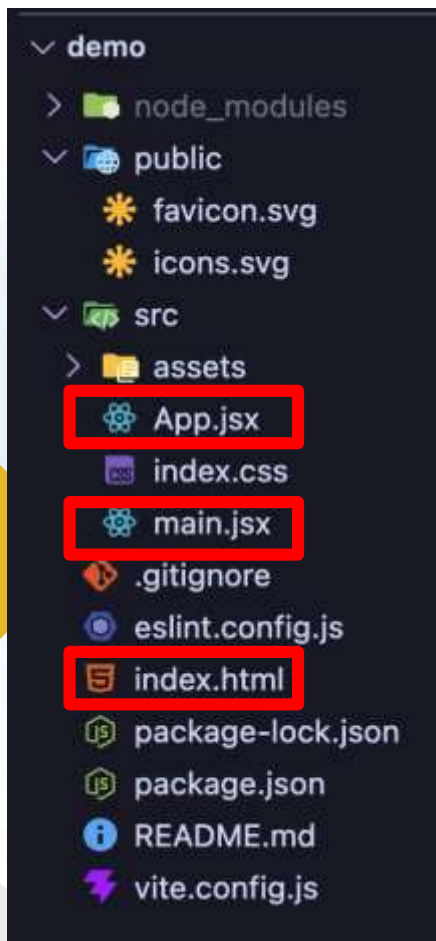
Anatomy of ReactJS

CSIT340 – Industry Elective 1
Mr. Lance Vincent A. Salera

ReactJS

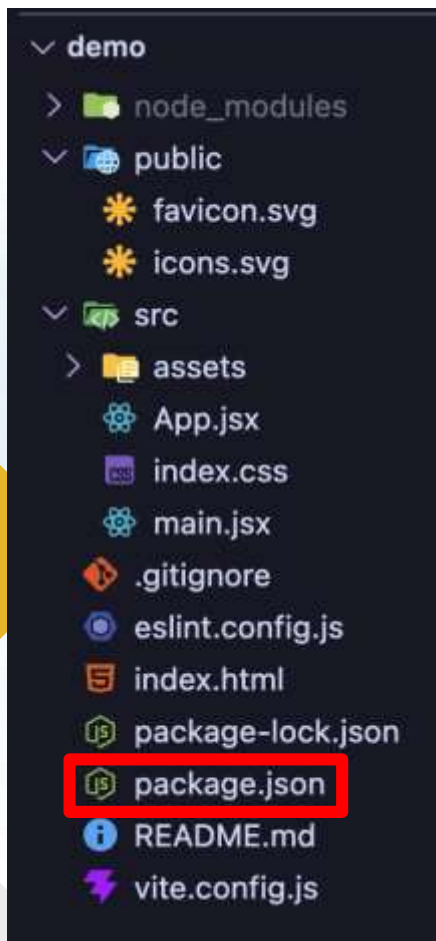
- Updating a page in vanilla JavaScript **requires** naming the node at every place the data changes.
- The developer writes what the page contains for a given set of **data** and writes no instruction to locate or modify a node.
- Locating a node is not absent from a React project; it occurs **once**.

ReactJS: anatomy



- A React project contains files **generated by tooling** rather than written by hand.
- Three of them contain everything the application does: **index.html**, **src/main.jsx**, and **src/App.jsx**.
- The remaining entries are **configuration** and installed code.

ReactJS: anatomy

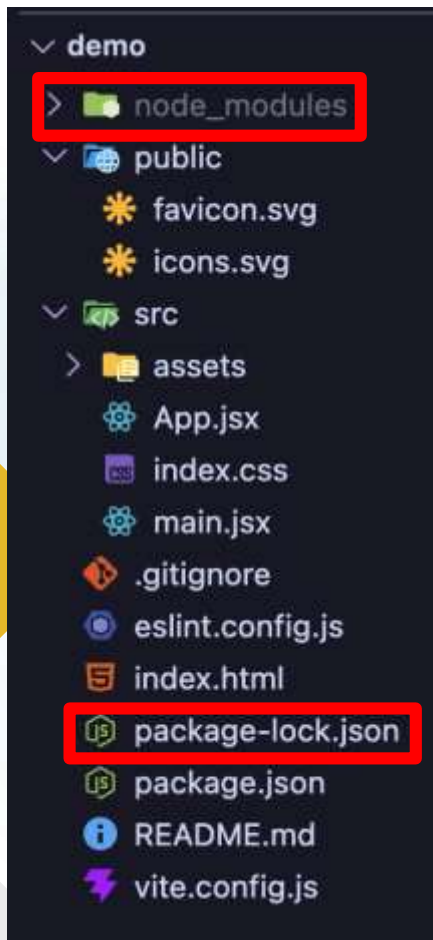


- **package.json** declares which packages the project requires and which commands it supports.

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint .",  
  "preview": "vite preview"  
},
```

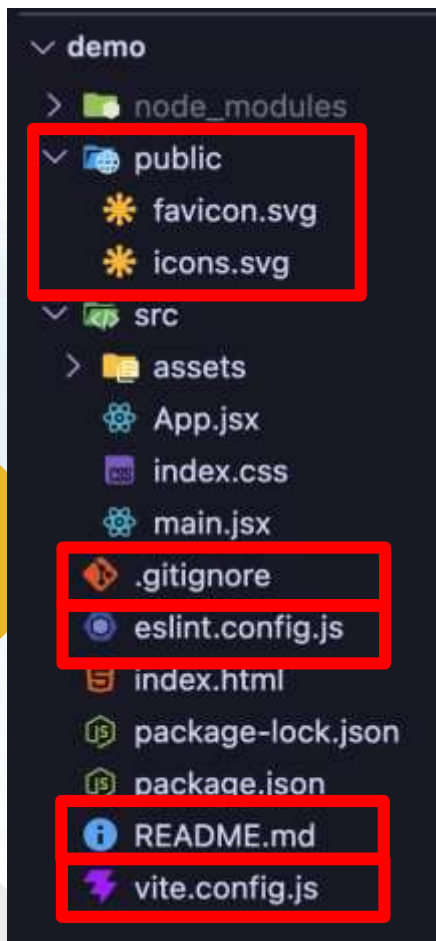
```
"dependencies": {  
  "@tailwindcss/vite": "^4.3.3",  
  "react": "^19.2.8",  
  "react-dom": "^19.2.8",  
  "tailwindcss": "^4.3.3"  
},
```

ReactJS: anatomy



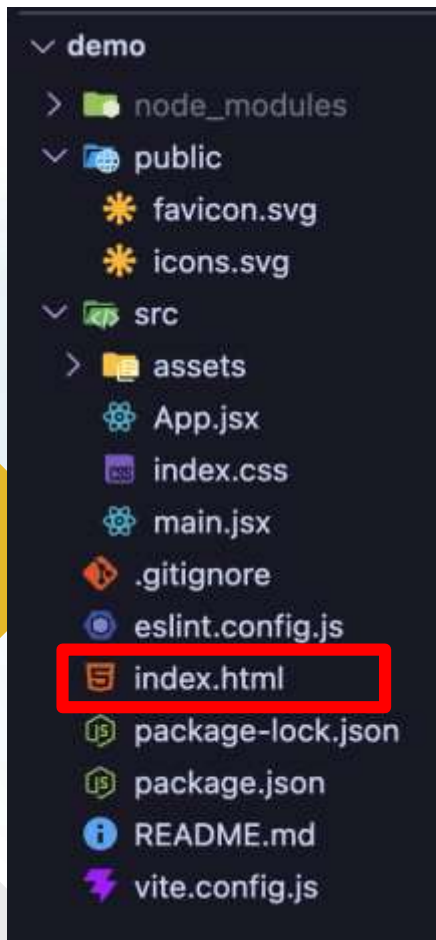
- **package-lock.json** records the **exact version** of every installed package, so an install reproduces identically on another machine.
- **node_modules** holds the installed packages themselves and is generated rather than written.

ReactJS: anatomy



- **vite.config.js** and **eslint.config.js** configure the tools that process the project.
- **.gitignore**, **README.md**, and **public/** concern version control, documentation, and static files.
- Only **one** of these files is an **HTML document**.

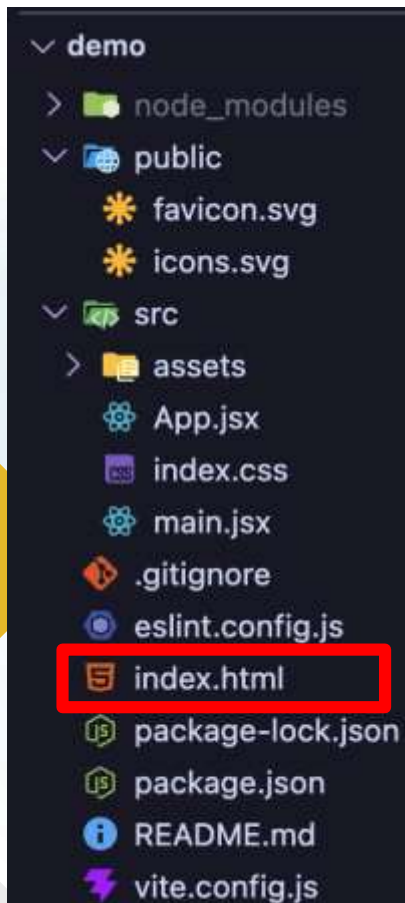
index.html



- The file sits at the **project root** rather than inside **src/**.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>demo</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

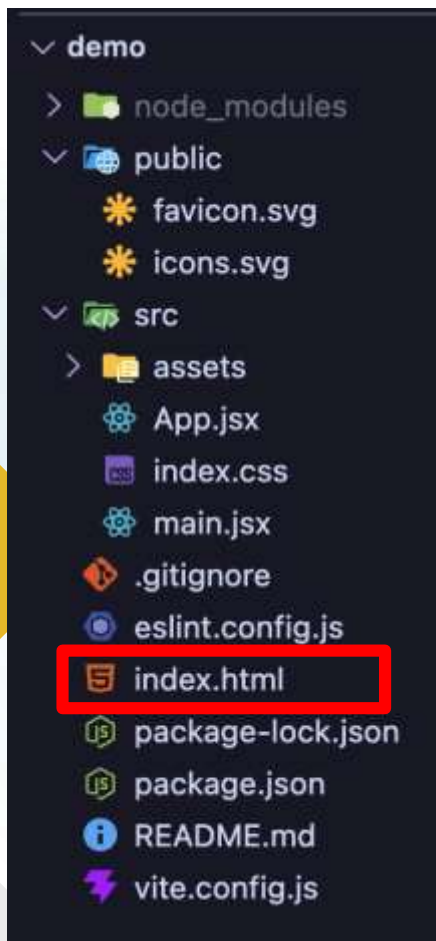
index.html



- The **body** contains **two elements** and no text content.

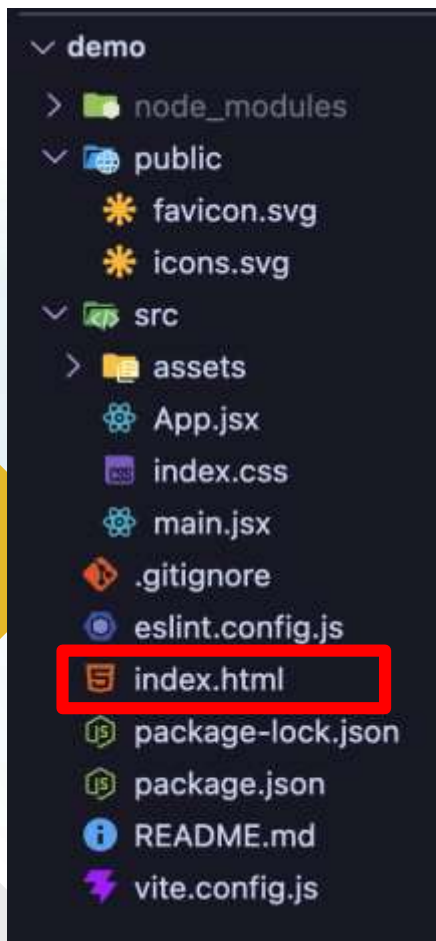
```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>demo</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

index.html



- The **heading**, the **paragraph**, and **every element** the user sees **appear nowhere** in this file.
- A vanilla project holds its **content** in the **document** and uses **scripts** to modify that content.
- This document holds **no content** to modify.

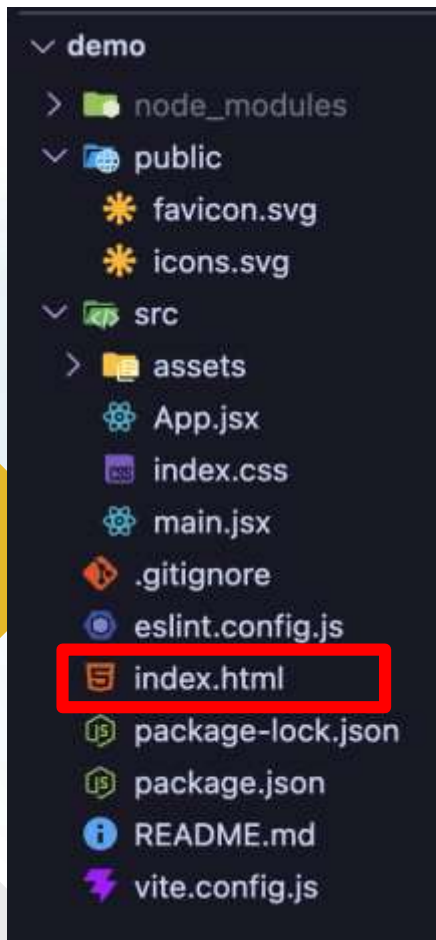
index.html



- One of the two elements in the body is a **script**.
- The other is a **div** that is empty on purpose.

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

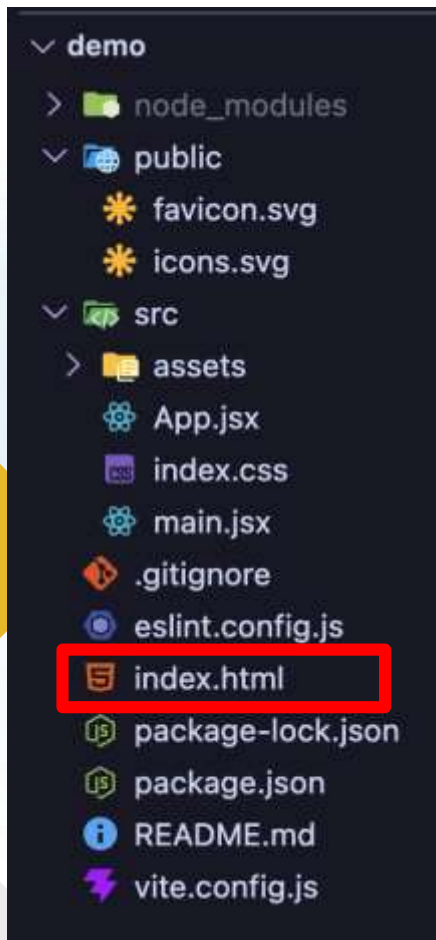
index.html: root



- `<div id="root"></div>` is **empty** on disk and stays empty until **JavaScript** runs.
- The element exists so that later code has a **node** to locate.

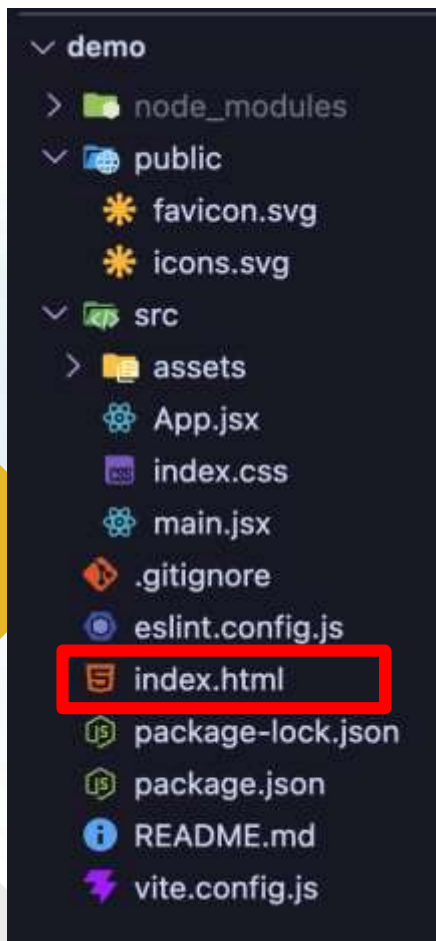
```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

index.html: root



- Elements written outside the **div** are **unaffected** by React.
- The **div** is the **region** of the page React is given, and the rest of the document stays under the author's control.
- **Something** has to find this **node** before anything can be placed inside it.

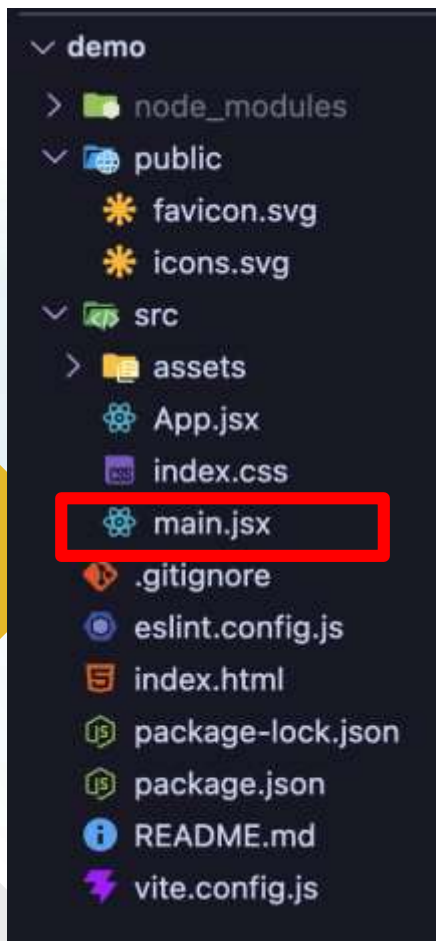
index.html



- The script element loads **src/main.jsx**.
- **type="module"** makes the browser treat the file as a **module**, and a module script does not run until the document has been parsed.
- A script marked this way never runs before the **div** exists.

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

main.jsx



- Contains **four imports** and **one statement**.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

modules

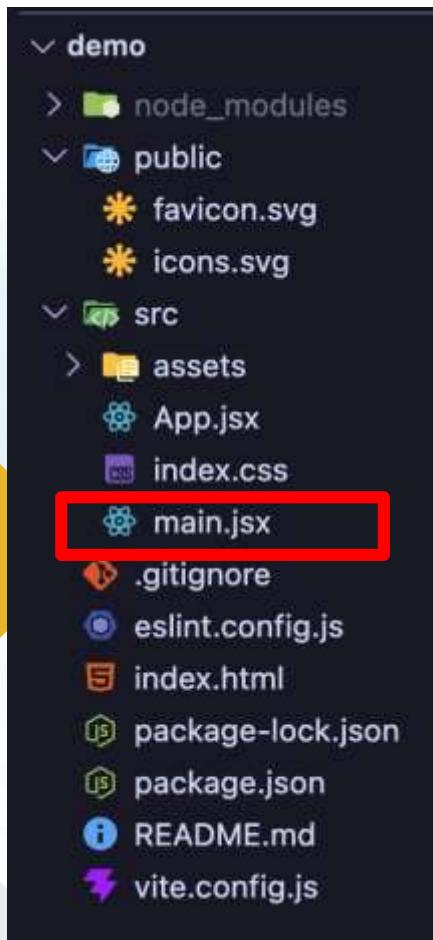
- A **JavaScript** file's **top-level names** are ordinarily **shared** with every other script on the page; a **module's** are **private** to itself.
- A module states which values it takes **from other files** and which values it offers **to them**.
- **export** marks a value as available to other files.
- **import** names a value and the file it comes from.

modules

- A **module** cannot run **until every file it imports** has been **fetch**ed and **evaluated**.
- A module's imports are **fetch**ed while the document is still being **parsed**, and the module **runs** only after parsing is complete
- **Deferral** is a consequence of allowing files to **import** each other, not a separate feature of the attribute.

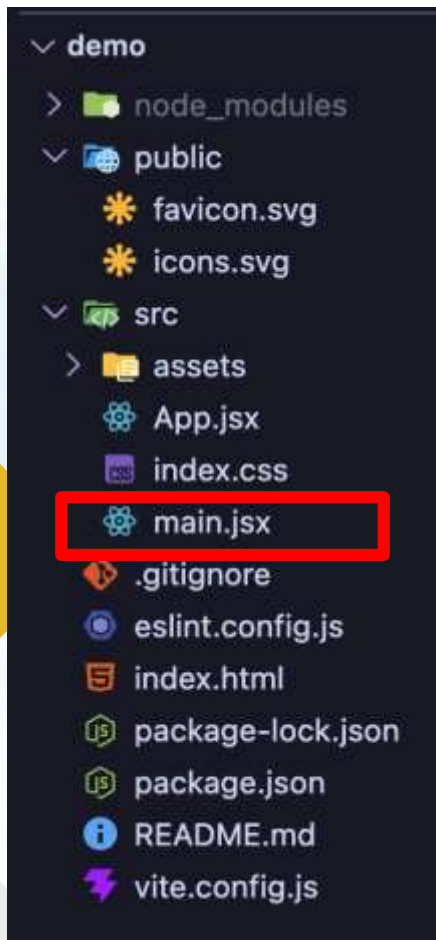
main.jsx: getElementById('root')

- **document.getElementById('root')** returns the **empty div** declared in **index.html**.



```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
)
```

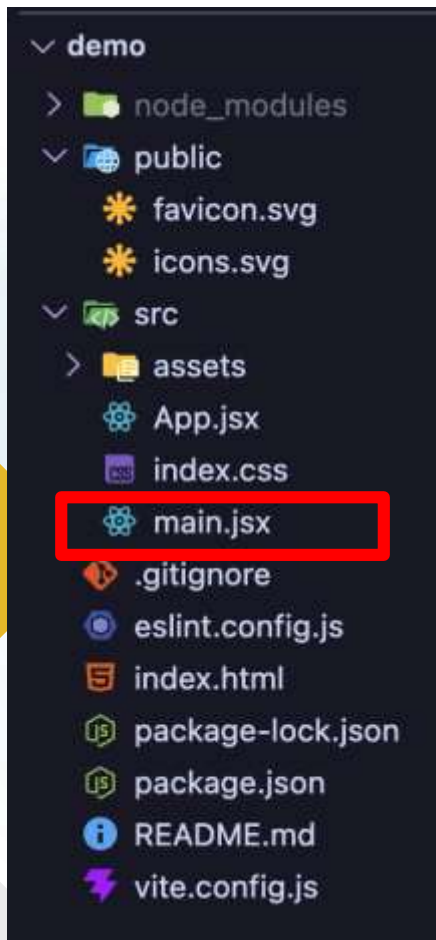
main.jsx: getElementById('root')



- The call belongs to the **browser** and is available to any script, with or without **React**.
- This is the only lookup the application's own code performs.

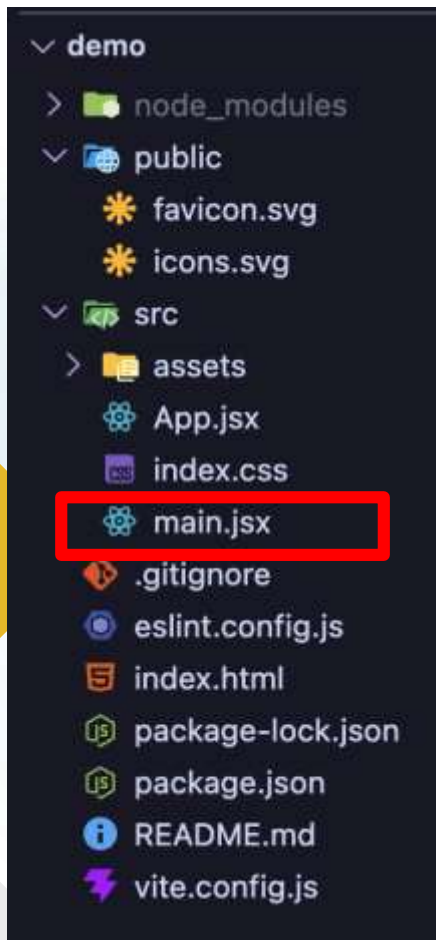
```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
)
```

main.jsx: getElementById('root')



- The string '**root**' is the only name the **HTML** and **JavaScript** file must agree on.
- Changing the **id** in one file without changing the other **breaks** the application at this line.
- The node is located, and then it is handed to **something else**.

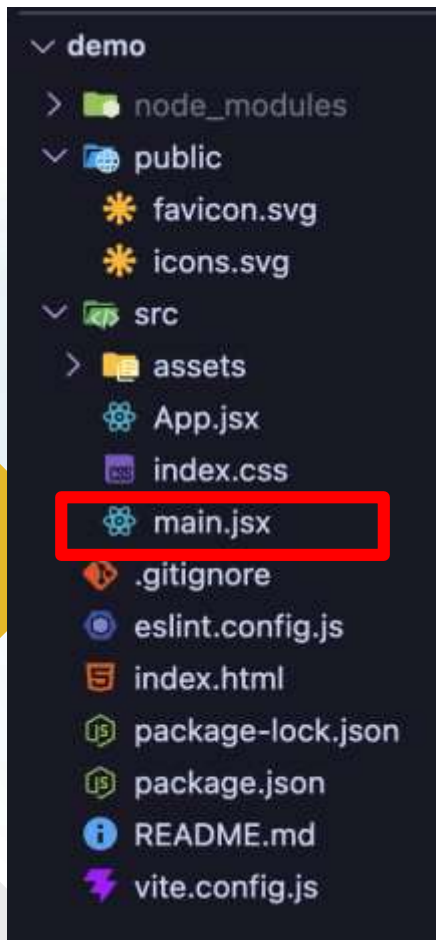
main.jsx: createRoot()



- **createRoot()** accepts one DOM node and returns a **root** object.
- React treats the contents of that node as **its own** from this point forward.
- **createRoot()** displays nothing, and calling it alone leaves the page blank.

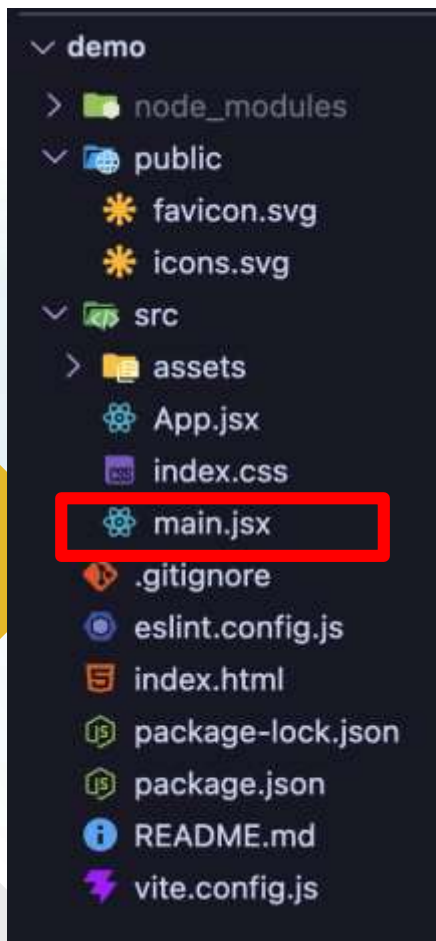
```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

main.jsx: render()



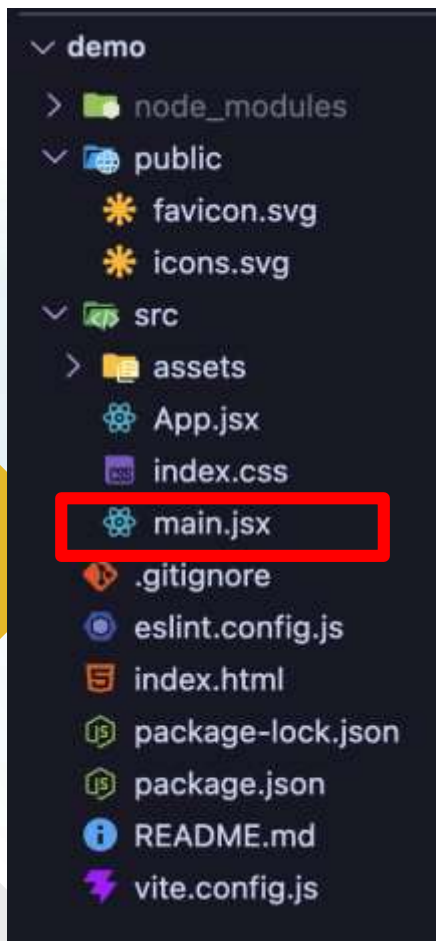
- **render()** is called on the object **createRoot()** returned.
- **render()** accepts a **description** of what the **page** should contain and displays it inside the root.
- Calling **render()** again causes **React** to compare **descriptions** and apply only the differences.

main.jsx: JSX



- The argument to **render()** is written as **tags**, inside a **JavaScript** file.
- **Tag syntax** written inside **JavaScript** is called **JSX**.
- **JSX** is **not HTML** and is **not a string**, and it is **converted** into **JavaScript** before the browser receives it.

main.jsx: JSX



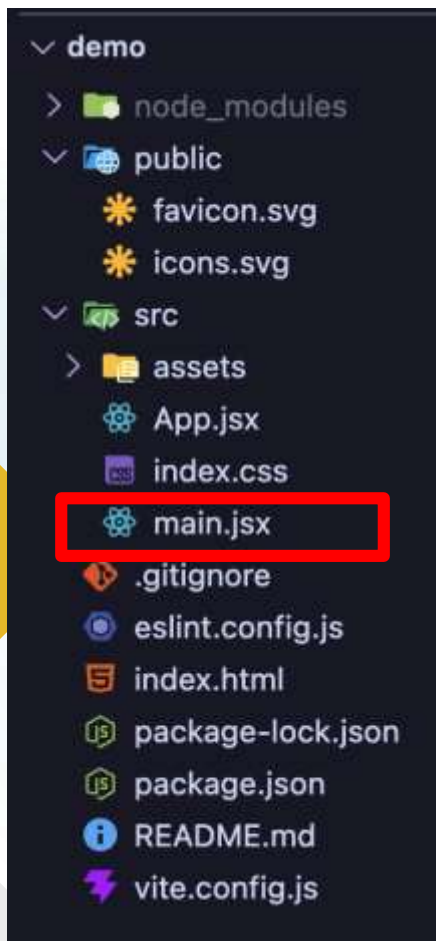
- The **JSX** passed to **render()** is a **tree of objects** describing what the page should contain.
- **App** and **StrictMode** are written as **JSX tags**.

```
createRoot(document.getElementById('root')).render(  


```
<StrictMode>
 <App />
</StrictMode>
```

  
)
```

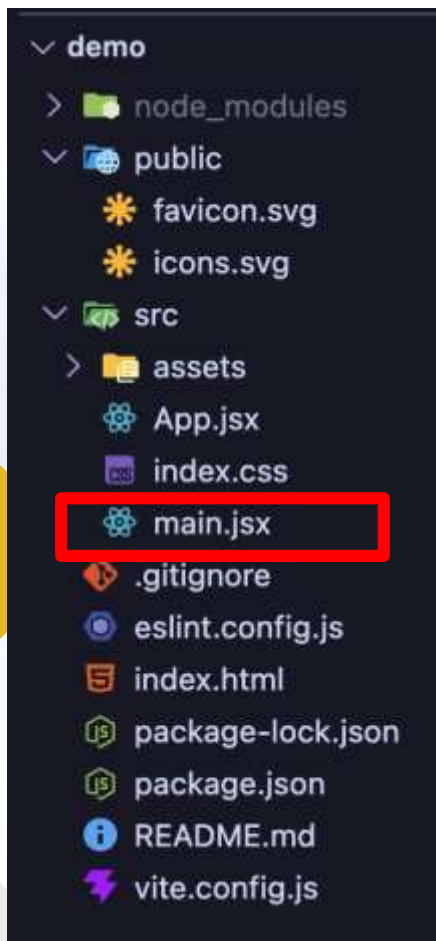
main.jsx: imports



- **StrictMode** is imported from **react**, and **createRoot** is imported from **react-dom/client**.
- **react** contains the part that produces **descriptions**, and **react-dom** contains the part that writes to the **DOM**.
- **createRoot** is in **react-dom** because it takes a **DOM node**.

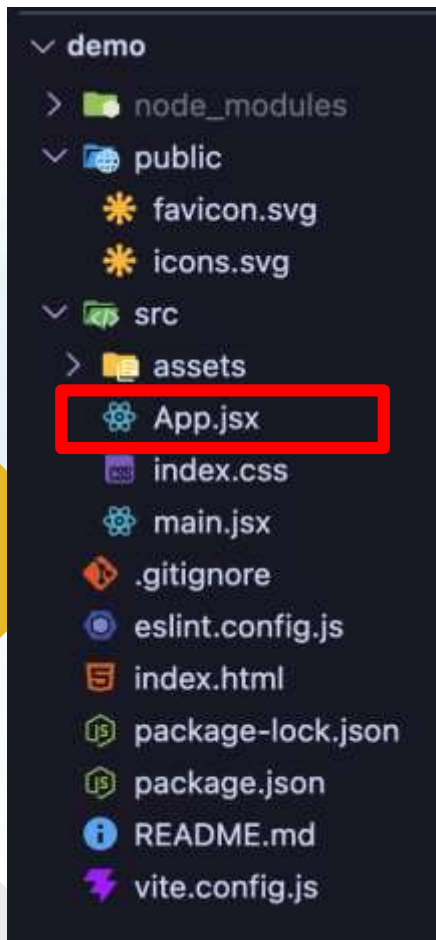
```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'
```

main.jsx: StrictMode



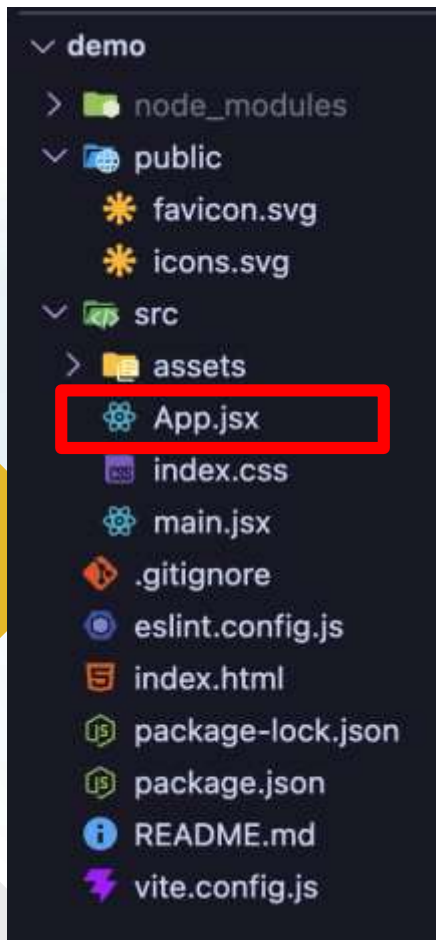
- **StrictMode** displays nothing of its own and produces **no node** in the **DOM**.
- **StrictMode** causes **React** to run certain code **twice** while the project is in **development**, so that code which depends on running once **fails visibly**.
- **StrictMode** has no effect in a production build.

App.jsx



- The file declares one function named **App** and exports it.
- App returns **JSX** and is called nowhere in the file.

App.jsx



- **App** is the name written as a tag in **main.jsx**.
- React calls **App** and displays the **JSX** it returned inside the **root div**.

```
function App() {  
  return <div>Hello World!</div>;  
}  
  
export default App;
```